

What is the most efficient algorithm to sort data in python

Heath Nowell

Table of Contents

Abstract	Page 3
Introduction	Page 4
Literature Review	Page 5
Procedure	Page 6
Analysis	Page 7
Discussion	Page 9
Conclusion	Page 10
Acknowledgments.....	Page 11
Bibliography	Page 12

Abstract

The experiment was to study how writing efficient & inefficient code affects code speed because when coding comes to the real world, sorting data is constantly happening, within milliseconds automatically, and efficiency is key. So, finding out if efficient code is key is important

The problem is some programmers write inefficient code, which can lead to problems. If the algorithm used libraries and effective ways to solve the problem, then the amount of time the code took to finish will be minimal.

First, a free dataset was found. Then the control group was tested using Excel. Then, 3 different programs sorted the dataset and ran each program 3 times and made sure the dataset was correctly sorted, which the program returns a precise time taken for the program to finish. Lastly, data was documented.

The results of the experiment were surprising. From looking at the time taken to sort the dataset, the code that was deemed “inefficient” outperformed the other codes which were deemed “efficient” and “sorta-efficient sorta-inefficient”. The ineffective code was nearly 3 times faster than the other codes.

From the results, the conclusion is that inefficient code should be used to sort SMALL AND MINIMAL DATA. The big library and smaller library that sorted the code slowed down the program since the data was minimal which is why the times are similar. The hypothesis was rejected for this, but if a big experiment using big data and constant edits of data, the hypothesis might’ve been different

Introduction

- A. **Experimental Problem** - What is the most efficient algorithm to sort data in python?
- B. **Hypothesis/Engineering Goal** - If the algorithm uses libraries and effective ways to solve the problem, then the amount of time the code takes to finish will be minimal
- C. **Rationale** - The experiment is important because the sorting of datasets, dictionaries, and lists, strings, and sets is important when it comes to storing data. Making sure the algorithm being used is the most effective and minimal memory is taken is a big deal. Some algorithms being used are not effective and take up a decent chunk of memory and are a massive waste of time and memory because of the programmer not understanding how to write fast and effective code. This is a modern-day problem of coding because not all programmers are good at writing effective code, instead, write messy code that will one day bring a big impact all because of poor code practice. This is important because the experiment will show how at the beginning of the experiment the code that was written to solve the problem would work but would be super inefficient.

Literature Review

From research on the 5 websites supporting the experiment, all say the same thing. Writing efficient code is essential. Not a single website on the internet says it doesn't matter. By writing inefficient code you're scaling the software issue to an infrastructure problem, for example, one big-data process that takes more than 24 hours of execution time (runtime), for the same inputs, after optimization runtime can take only 90 minutes. (Arc BR, 2021). This is why the experiment is putting to the test of does writing efficient code really matters.

Can optimization of code really save time? From research it looks like writing efficient code matters, as a non-expert programmer, it can be difficult to know how to optimize a particular fragment of code so that it executes more rapidly, beyond the use of profilers and analysis tools to detect bottlenecks or parts where the program spends most of its execution times. In this sense, it is important to highlight that an inefficient piece of code can waste a significant amount of time in the long term when repeated millions of times. (Corral-García, J., González-Sánchez, J.-L. and Pérez-Toledano, M.-Á., 2018).

This is why programmers who are inexperienced can write inefficient code which will cause problems later on for the company that hired the programmer This is why spending the information about efficient code and the research behind efficient code is important as so many programmers write inefficient code. The fact that writing efficient code can save a ton of time is super important nowadays since so many things use software and writing inefficient code can waste a significant amount of time.

Efficiency in coding is very important. Efficient coding allows you to speed up your development time and makes your code easier to read, debug and maintain. (West, Richard, 2017). When a programmer writes efficient code then if another programmer was to look at the code/expand on the code if the code is efficient then the programmer won't have to worry about the code already written and debugging will be way easier and development time will be sped up.

Procedure

Variables:

Independent Variable: The code (written code / written algorithm)

Dependent Variable: The speed of the program

Control Group: A human looking through the dataset and sorting the information

Hypothesis/Engineering Goal: If the algorithm uses libraries and effective ways to solve the problem, then the amount of time the code takes to finish will be minimal as well as minimal memory usage

Location of Experiment: Home

Materials:

- 1 dataset of choice (for this experiment the dataset of choice will be the top 950 highest-grossing movies of all time)
- 1 computer/laptop
- 1 Coding software

Procedures:

1. Got The top 950 highest-grossing movies of all time dataset
2. Learned how to code in python
3. Tested the control group by manually sorting the whole dataset
4. Created different algorithms that will sort the dataset, but each will use different logic used to sort the dataset (creates varying code speed, logic, and efficiency).
5. Ran each program and the program will take in the dataset, sort the dataset, and produce and new dataset. Also returns a precise time taken for the program to finish. Ran each program 3 times for accurate results.
6. Analyzed results and drew conclusions

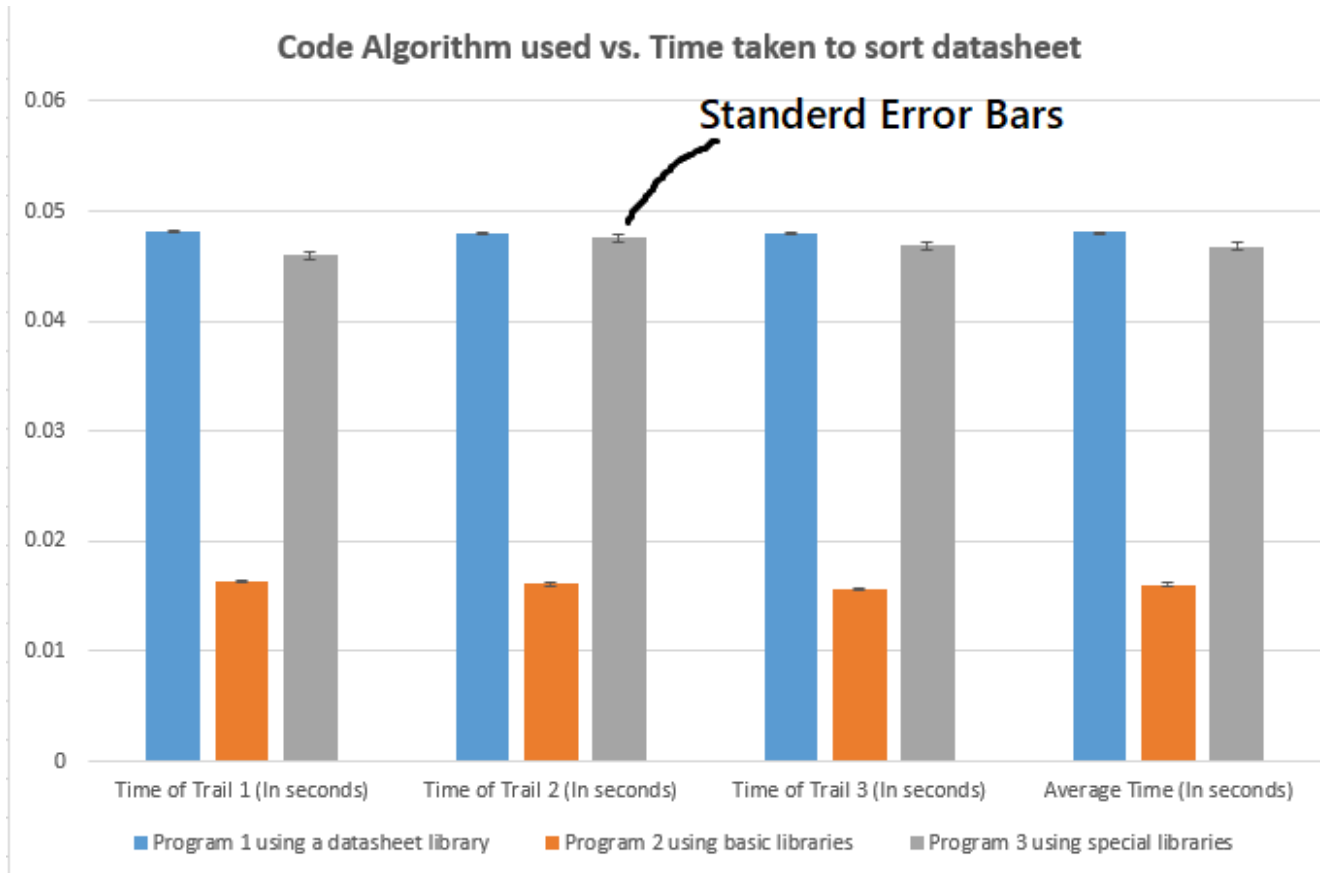
Analysis

Data Table:

Code Algorithm used vs. Time taken to sort dataset

Different programs	Time of Trail 1 (In seconds)	Time of Trail 2 (In seconds)	Time of Trail 3 (In seconds)	Average Time (In seconds)
Program 1 using a dataset library	0.0482351779	0.0479927062	0.0480020046	0.0480766295
Program 2 using basic libraries	0.0163722038	0.0161294937	0.0156333446	0.016045014
Program 3 using special libraries	0.0459783077	0.0475444793	0.046860218	0.046794335
Human manually sorting dataset	11.65	11.28	9.68	10.87

Graph:



Results:

In the chart, The lines are indicated by color, meaning if we look at the graph program 2 takes the least amount of time to sort the dataset. For the experiment, the program ran 3 times giving 3 trails, and then from the trails, the average time was calculated for each program. So, looking at the data, the chart shows that using basic libraries is 2 times faster than using a dataset library or a special library. The human sorting dataset is not included because the time would go way off the graph and not give accurate results since the time is nearly 200-500 times higher than the programs sorting the dataset. To sort the dataset, the app Excel was used.

Discussion

The hypothesis was supported because of the amount of research on how effective code affects time. The experimental errors were the fact that the experiment didn't have access to high amounts of data, servers, and money to be able to test the speed for sorting big data in small amounts of time, in a loop. If given more money and time, this would be able to be corrected and a new experiment could be conducted with more promising results. To minimize bias, nothing was judged. The data in the trials align. This means that the experiment gave accurate results. This meant only 3 trials were necessary for average. The only really hard variable to maintain was the computer hardware, as better parts would mean faster times. The outcome of the experiment was not affected. The results compared to commonly held beliefs were actually the same. Online website show libraries are better for small data management (like really small) the result would be the same. There was a good amount of data, but not enough for a full conclusion. If the project was repeated, the project instead would be comparing if libraries are better for code efficiency, otherwise, the project would be the same.

Conclusion

In the results, the program using basic libraries was 3 times faster than the other 2 using special libraries. There was a negative correlation between the variables as when the efficiency of the code was increased, the time taken was increased. The reasoning is the big library and smaller library that sorted the code slowed down the program since the data was minimal, which is why the times are similar. The hypothesis was rejected for this. If anyone was to look at how code is used in the real world, the code isn't just some minimal lines of code to sort a small amount of data. Instead, data has to be constantly sorted in milliseconds, and then stored, which is why using big libraries, is good. Because for one not as much code is needed to be written and two, libraries save time, both in writing and in executing. In general, using libraries for a simple task such as sorting a simple dataset is not efficient. But, for complex things, such as sorting multiple and big lengths of data at once, libraries are a lifesaver. In conclusion, simple algorithms are better off without libraries, and complex algorithms practically need libraries to be efficient. The next logical step would be to actually get a server and data that would be constantly exchanged and sorted, to actually see if libraries have a big effect on code efficiency. That would require money and time, as getting servers and big datasets and all is hard and needs money. These results could show that the more complex the algorithm or problem to solve, that libraries would be efficient and time-saving.

Acknowledgments

Thank you to my parents for supporting me in this experiment.

Bibliography

BR, A. (2021) Why to write efficient code? LinkedIn. Arc BR. Available at: <https://www.linkedin.com/pulse/why-write-efficient-code-arcenio-brito> (Accessed: October 19, 2022).

Apotheon, B. (2011). Why clean code is more important than efficient code. *TechRepublic*. <https://www.techrepublic.com/article/why-clean-code-is-more-important-than-efficient-code/> (Accessed: November 29, 2022).

Karlsruher Institut für Technologie (KIT). (2021, May 25). Scientific software - Quality is not always good. ScienceDaily. Retrieved October 19, 2022, from www.sciencedaily.com/releases/2021/05/210525113715.htm

Corral-García, J., González-Sánchez, J.-L. and Pérez-Toledano, M.-Á. (2018) Evaluation of strategies for the development of efficient code for Raspberry Pi devices, Sensors (Basel, Switzerland). MDPI. Available at: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6263706/> (Accessed: October 19, 2022).

West, R. (2017, September 12). *Efficient Code*. WordPress. LearnLearn. <https://learnlearn.uk/alevelcs/efficient-code/#:~:text=Writing%20Efficient%20Code,of%20loops%20for%20repeated%20actions> (Accessed: November 29, 2022).